1 Background and Motivation

Code review is the principal human quality gate for software changes. It serves purposes beyond defect detection, such as knowledge transfer and shared norms [1], and low review coverage is associated with higher post-release defect-proneness [2]. In open-source projects its cost falls on a small number of maintainers, and coding agents that open pull requests at a rate no maintainer can match add to it: verification, rather than authorship, becomes the bottleneck.

LLM reviewers could share this load, but in an industrial deployment developers reported faulty, unnecessary and irrelevant comments, and average pull-request closure time rose from 5 h 52 min to 8 h 20 min [3]. The costs are *noise*, *misses* and the effort of *confirming* each comment, which amounts to reviewing the change again. A failing test reduces that effort to running it, but whether a behaviour is a defect cannot always be settled from the code alone; by the test-oracle problem [4], the expected behaviour must come from a source other than the model.

This project develops **eyeful**, a multi-agent reviewer that aims to find the defects that matter, report little else, and attach evidence a maintainer can check. It falls within the supervisor's topic of LLM-based and human analysis of bugs in open-source software.

2 Related Work and Positioning

LLM-based review can be improved at the *model* layer, by pre-training on review data as in CodeReviewer [5], which needs data and compute beyond an individual project, or at the *harness* layer, where the design of the agent's tools and workflow alone substantially changes outcomes [6]. eyeful works at the harness layer, reusing models as they are.

eyeful runs the core library of **pulls.review**[1] unmodified to group a change by intent, and reimplements the staged pipeline of **revmux**[2], whose findings are validated by an agent that reads the code. It adopts benchmark acceptance criteria for executed evidence [7], [8], [9], which LLM bug reproduction [10] has applied to reported bugs but not to candidate review findings; review benchmarks [11], [12] match findings to human-confirmed comments with an LLM judge and execute none. eyeful's contributions are per-change expert planning, oracle-graded execution of findings and a controlled evaluation.

3 Aim, Research Questions and Objectives

Aim. To determine whether planned multi-expert review with execution-based verification finds more of the defects that matter, with less noise and with evidence a maintainer can check, than simpler LLM reviewers, and at what cost.

- **RQ1 (detection).** Does a planned panel of experts find more known defects, and more categories of problem, than a single agent using the same model and skills?
- **RQ2 (noise).** What proportion of reported findings is valid under each verification condition?
- **RQ3 (verifiability and cost).** How many findings can execution reproduce and fix, how many genuine defects does it fail to confirm, and at what cost in tokens, money and time?

No direction is assumed for RQ2; if read-only findings are already mostly valid, execution's value lies in RQ3. The objectives are to:

1. Review the literature on code review, LLM test generation, the test-oracle problem and LLM-as-a-judge.
2. Complete the research core of eyeful (planner, experts, verifier, read-only judge and run archive) and a single-agent baseline.
3. Prepare subjects from three public benchmarks and automate their scoring.
4. Run the main experiment and score it.
5. Ablate the planner and oracle grading, and analyse cost.

[1] github.com/antfu/pulls.review

[2] github.com/umputun/revmux

4 Methodology

4.1 Review Workflow

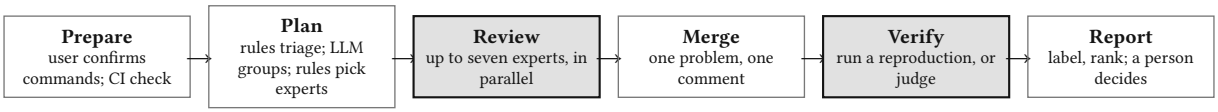


Figure 1: The review workflow. Shaded stages are those varied or measured in the experiment.

A review has six stages (Figure 1). The agents decide what to look at; plain code owns the order of the stages, the budget and when to stop.

1. **Prepare.** eyeful finds the project's commands from its manifests and runs them only after the user confirms them. The lint command is the CI check: if it fails, the change gets a diagnosis instead of a review. The test suite runs on the change, and on the base commit where it fails.
2. **Plan.** Rules set aside lockfiles and generated or vendored files. The planner groups the rest by intent, and a fixed table picks the experts each group needs.
3. **Review.** Up to seven experts read their groups in parallel with read-only tools. They work independently, so agreement counts as corroboration. Each reports every problem the change introduces, with a severity; `note` findings are not verified.
4. **Merge.** Findings on the same lines that one fix would resolve become one comment, which records how many experts reported it.
5. **Verify** and **Report.** `error` and `warning` findings are verified as described below. Comments are labelled and ranked; eyeful never approves a change, and a person decides.

Verification by execution. For a finding a unit-sized test could show, a separate agent writes a reproduction, which must pass the gates of Table 1 in order; other findings go to the judge.

Table 1: Gates a reproduction must pass, in order.

Gate	Requirement	Basis
Runs	The test builds and executes on a fresh copy of the change	[9]
Not flaky	All five runs end the same way	[9]
Fails as claimed	The output contains the failure text the test writes for the claimed reason, not, for example, a compile error	this work
Introduced	It does not fail with that text on the base commit; otherwise the finding is pre-existing and cannot block	[7]
Fail to pass	A fix by an agent that sees the finding and test, not the expert's reasoning, makes it pass without editing it	[8]
Pass to pass	Project tests that passed before the fix still pass	[8]

The first four gates show that the change introduced a defect; the last two show only that the fix is plausible, since agents wrote both it and the test. Neither shows that the test is right, so each reproduction declares its oracle in the taxonomy of Barr et al. [4], which the verifier checks: *implicit* (a runtime failure such as a crash, race or timeout), *specified* (a quotation found verbatim in the description, commits or documentation) or *derived* (an existing project test that now fails). Only these may block. A new test supported only by passing on the base commit, or by the model's reading (a *human* oracle), becomes a question to the author with the failing test attached. The checks catch a mislabelled oracle, not a quotation that fails to support the assertion.

4.2 Review Standards and the Judge's Criterion

Experts review against published standards (for security, OWASP and CWE) and *Agent Skills* from skills.sh[3], vendored unchanged and pinned by hash. Each skill belongs to one expert, and the single agent receives all of them, so both configurations receive identical guidance. Findings cite the skill or CWE identifier they rely on.

[3] skills.sh

The read-only judge calls a finding valid only if (a) the problem exists in the changed code, (b) it is introduced or exposed by the change, and (c) it alters observable behaviour or violates a stated contract. Execution tests the same claim, so the conditions differ in evidence, not in their definition of a defect.

4.3 Experimental Design and Analysis

Subjects. A subject is one change with known defects, from three public benchmarks. *SWE-bench Verified* [8]: about 30 tasks from seven Python projects, reverted so that the change removes the reference fix, with the issue text withheld; tests run in the task's official container without network access. Only this set has runnable tests, so it alone answers RQ3. *Code Review Bench* [11]: 50 pull requests with 173 human-confirmed comments. *ReviewBench* [12]: 50 of its 219 pull requests, sampled by language from the 194 not used in development. Runs whose agents read outside the repository under review are excluded and counted.

Procedure. The design is within-subject. Each subject is reviewed by the planned panel and by a single agent given every skill (RQ1), three times each. The archived findings of every run pass through each verification condition: none, read-only (an LLM judge [13]) and execution (RQ2, RQ3), so verification is the only difference; without runnable tests, only the first two are compared. The planner is ablated with two fixed experts (correctness and security), once on SWE-bench and Code Review Bench against the panel's first run, and oracle grading by rescoring archived reproductions with every one allowed to block; about 860 reviews are run. Every agent and judge, the benchmarks' official judges included, uses DeepSeek V4.1 Flash, so results are internal comparisons, not leaderboard scores. Prompts, budgets and this plan are fixed first.

Measures and analysis. Under *none* every finding is reported, under *read-only* those the judge does not dispute, and under *execution* those reproduced or, lacking a reproduction, not disputed. On SWE-bench the known defect is found when the official harness resolves the task with the fix agent's fix for a reported finding (its `FAIL_TO_PASS` and `PASS_TO_PASS` tests pass). Code Review Bench's official judge matches findings to golden comments under the default Core profile, and ReviewBench's official judge and classifier report grounded recall and augmented precision. For RQ1, a known defect or golden comment counts when two of a configuration's three runs find it; recall is compared with McNemar's exact test, also against the union of the single agent's three runs, costing about one panel run, and categories are the benchmarks' own labels. For RQ2, precision is the share of findings that resolve the SWE-bench task or match a Code Review Bench golden comment, and ReviewBench's augmented precision; conditions are compared with a paired bootstrap over subjects. For RQ3, the verifiable rate is the share of executed findings whose reproduction passes all six gates, and the verification loss the share of findings that resolve a SWE-bench task but execution does not confirm. Estimates carry 95% bootstrap intervals. The main threats are lenient LLM judges, incomplete golden sets, which make precision a lower bound, one model as reviewer, judge and scorer, SWE-bench predating the model's cut-off, with the removed code in the diff, and RQ3 resting on about 30 Python subjects.

5 Work Plan

Figure 2 places each task against the Moodle deadlines: final proposal (21 October), ethics clearance and data management plan (28 October), interim report (7 December), dissertation (14 April) and demonstration (13–14 May).

The research core is implemented and piloted, so the critical path runs from its stabilisation and the benchmark subjects, built in parallel, through pilot runs that freeze the prompts and budget, to the main experiment and its analysis. A pilot panel review takes about ten minutes, so the experiment fits February and early March. No task is shorter than two weeks, early December is kept light for other modules, and progress is reviewed at fortnightly supervision meetings.

The minimum deliverable is all five objectives on the three benchmarks. If the plan slips, the revmux comparison, an extension, is dropped and the ReviewBench sample halved; SWE-bench, which alone answers RQ3, is kept.

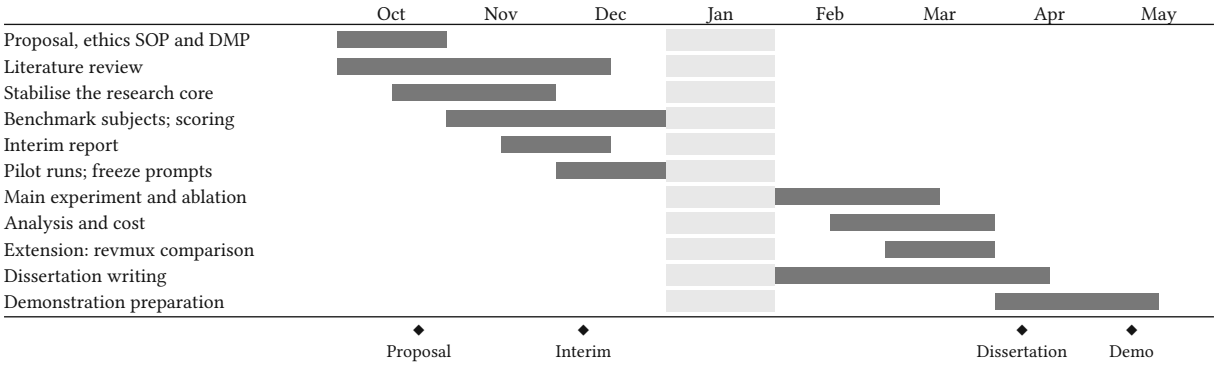


Figure 2: Schedule in half-month intervals; shading marks January examinations, ◆ deadlines.

Table 2: Principal risks and mitigations.

Risk	Mitigation
SWE-bench tasks will not build	Use each task's own image; replace failures from the same projects.
Model cost exceeds budget	Per-review budgets; exhausted reviews return verified findings as partial.
Model or API access lost	The model sits behind one adapter; every run is archived, so analysis continues offline.
Tuning leaks into results	Fix everything before the main run; never sample ReviewBench's 25 test pull requests.
Scope creep	Defer all clients beyond the research core; only the five objectives are required.

6 Ethics and Professional Issues

The study involves no human participants, and no output is labelled by hand. The benchmarks hold public code and review comments, used under their licences without identifying any developer, and only public code reaches model providers. The study is minimal risk; the ethics SOP and data management plan will be agreed with the supervisor and submitted by 28 October. Only confirmed project commands run, and a person, not eyeful, decides on every change. The work supports UN Sustainable Development Goal 9 (infrastructure).

Bibliography

[1] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, 'Modern Code Review: A Case Study at Google', in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2018, pp. 181–190.

[2] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, 'An Empirical Study of the Impact of Modern Code Review Practices on Software Quality', *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146–2189, 2016.

[3] U. Cihan *et al.*, 'Automated Code Review In Practice', in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 425–436.

[4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, 'The Oracle Problem in Software Testing: A Survey', *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[5] Z. Li *et al.*, 'Automating Code Review Activities by Large-Scale Pre-training', in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 1035–1047.

[6] J. Yang *et al.*, 'SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering', in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 50528–50652.

[7] R. Just, D. Jalali, and M. D. Ernst, 'Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs', in *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440.

[8] C. E. Jimenez *et al.*, 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[9] N. Alshahwan *et al.*, 'Automated Unit Test Improvement using Large Language Models at Meta', in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, 2024, pp. 185–196.

[10] S. Kang, J. Yoon, and S. Yoo, 'Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction', in *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

[11] Martian, 'Code Review Bench'. [Online]. Available: https://github.com/withmartian/code-review-benchmark

[12] GitHub, 'ReviewBench: An Open, Reproducible Benchmark for AI Code Review'. [Online]. Available: https://github.com/review-bench/ReviewBench

[13] L. Zheng *et al.*, 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena', in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023, pp. 46595–46623.